

# AI4IO: A suite of AI-based tools for IO-aware scheduling

Michael R Wyatt<sup>1,2</sup>, Stephen Herbein<sup>2</sup> , Todd Gamblin<sup>2</sup> and Michela Taufer<sup>1</sup> 

The International Journal of High Performance Computing Applications 2022, Vol. 36(3) 370–387  
© The Author(s) 2022  
Article reuse guidelines:  
[sagepub.com/journals-permissions](https://sagepub.com/journals-permissions)  
DOI: 10.1177/10943420221079765  
[journals.sagepub.com/home/hpc](https://journals.sagepub.com/home/hpc)



## Abstract

Traditional workload managers do not have the capacity to consider how IO contention can increase job runtime and even cause entire resource allocations to be wasted. Whether from bursts of IO demand or parallel file systems (PFS) performance degradation, IO contention must be identified and addressed to ensure maximum performance. In this paper, we present AI4IO (AI for IO), a suite of tools using AI methods to prevent and mitigate performance losses due to IO contention. AI4IO enables existing workload managers to become IO-aware. Currently, AI4IO consists of two tools: PRIONN and CanarIO. PRIONN predicts IO contention and empowers schedulers to prevent it. CanarIO mitigates the impact of IO contention when it does occur. We measure the effectiveness of AI4IO when integrated into Flux, a next-generation scheduler, for both small- and large-scale IO-intensive job workloads. Our results show that integrating AI4IO into Flux improves the workload makespan up to 6.4%, which can account for more than 18,000 node-h of saved resources per week on a production cluster in our large-scale workload.

## Keywords

High performance computing, workload managers, parallel file systems, job scheduler, IO contention, IO prediction, IO-intensive workloads

## 1. Introduction

Production workload managers (i.e. software systems for resource management and job scheduling) such as Slurm manage nodes and on-node resources, including processors and GPUs, but do not handle the wider set of off-node resources such as IO and network. Data-intensive jobs can end up in contention (i.e. competing for IO and network resources) and consequently may experience performance degradation. In other words, data-intensive jobs may take longer to run and may even fail to complete within their allocated time because of an IO or network wait time. In the best case contention scenario, jobs slow down on an expensive resource; in the worst case, they do not complete within the time assigned to them, wasting the entire resource allocation.

When dealing with IO contention on HPC systems, traditional resource managers and job schedulers have to manage complex IO pipelines, with local and global bottlenecks that can manifest at any time. Two common causes of IO contention are: (i) bursts of IO demand from concurrently running jobs and (ii) Parallel File System (PFS) performance degradation. Emerging IO-aware workload managers have the potential to prevent and mitigate the effects of IO contention caused by both bursts and PFS degradation. In other words,

through scheduling decisions generated with a priori knowledge of job resource usage, IO-sensitivity, and contention detection, IO-aware workload managers can execute jobs while preventing and mitigating IO contention. Tools that augment production workload managers with knowledge of IO contentions before and during the job executions are still missing. These tools must be able to: (i) identify and exploit temporal, system-specific patterns in resource management; (ii) augment job schedulers to be resource-aware; and (iii) avoid the need to rely on the users' feedback on resource utilization.

Our solution is AI4IO (i.e. AI for IO), a suite of AI-based tools that capture trends in historical HPC data and make

<sup>1</sup>Department of Electrical Engineering and Computer Science, University of Tennessee Knoxville, Knoxville, TN, USA

<sup>2</sup>Center for Applied Scientific Computing, Lawrence Livermore National Laboratory, Livermore, CA, USA

### Corresponding author:

Michela Taufer, Department of Electrical Engineering and Computer Science, University of Tennessee Knoxville, 401 Min H. Kao Bldg, 1520 Middle Drive, Knoxville, TN 37996-2250, USA  
Email: [taufer@utk.edu](mailto:taufer@utk.edu)

predictions that schedulers can use to prevent and mitigate IO contention, ultimately improving a system's resource utilization. Our suite currently consists of two tools: PRIONN and CanarIO. These two tools provide complimentary prediction services: PRIONN prevents IO contention by providing workload managers with accurate predictions of individual runtime and IO usage before each job execution, and CanarIO detects and mitigates IO contention by predicting the occurrence of IO degradation in real-time and predicting which jobs are IO-sensitive. We show how the use of the two tools in concert to support Flux, a cutting-edge production workload manager, can improve performance on HPC systems.

We provide a comprehensive approach to IO-awareness in Flux, preventing IO contention and mitigating the impact of IO contention both separately and in concert. To prevent the occurrence of IO contention, we use PRIONN, which leverages a neural network to interpret whole job scripts and predict per-job runtime and IO resource in HPC systems. To mitigate the impact of occurring IO contention, we use CanarIO, which identifies the occurrence and severity of IO-driven performance degradation events and the jobs sensitive to IO degradation. To make schedulers fully IO-aware, we use the two tools in concert on both a small- and large-scale system. We first extend the scope of the Flux simulator [Ahn et al. \(2020\)](#) to incorporate IO contention and IO-aware scheduling logic. Then, we develop decision logic pipelines for enabling IO-aware scheduling with PRIONN and CanarIO. Last, we apply AI4IO for resource-aware scheduling validating its correctness on a small-scale system and evaluating its effectiveness at preventing and mitigating IO contention on a larger, production system.

The main contributions of this work are as follows:

- We provide a concise overview of two tools, PRIONN and CanarIO, for preventing and mitigating IO bursts on HPC systems.
- We define a decision logic pipeline for enabling IO-aware scheduling with the support of both PRIONN and CanarIO.
- We extend the Flux scheduler to integrate IO-awareness through these two tools.
- We use PRIONN and CanarIO in concert on both a small-scale and large-scale system and empirically demonstrate how they improve the resource usage.

## 2. The challenges of performance loss in IO

### 2.1. Limited access to IO information

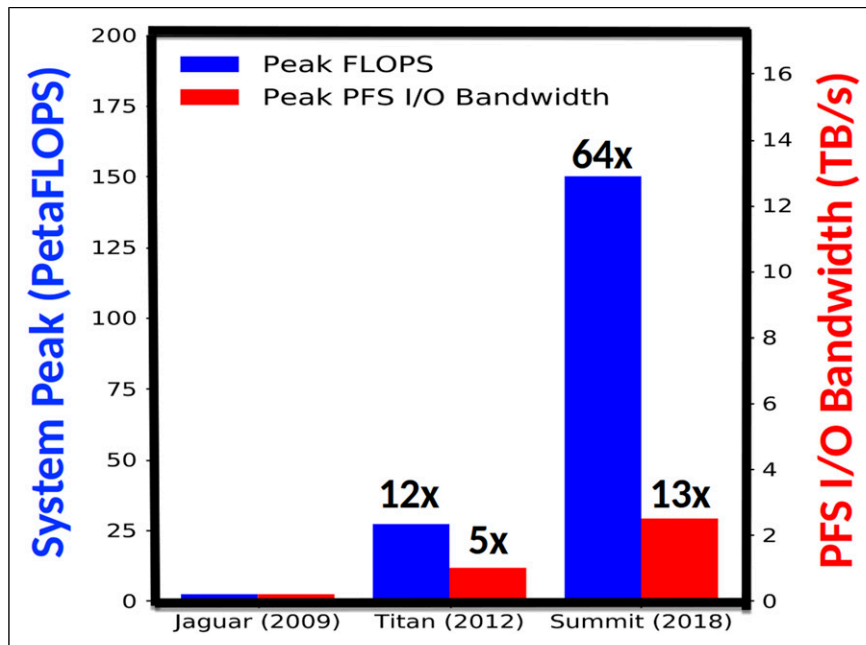
Current production batch schedulers are insufficiently equipped to access information on IO usage at the system level and predominately rely on information provided by users (e.g. number of number of nodes, the number of cores,

and the max time per job) for the job allocations. Delegating the specification of IO usage to users is not a feasible solution. Analysis of job traces in HPC centers shows how user-requested runtimes are often over-estimated [Wyatt et al. \(2017\)](#). For example, user-requested runtimes for nearly 3,00,000 jobs on the Cab cluster at Lawrence Livermore National Laboratory (LLNL) in 2016 had a mean error of 172 min [McKenna et al. \(2016\)](#). Furthermore, when extending the type of resources considered in scheduling decisions to include IO and other resources, users in scientific computing do not have an accurate understanding of job requirements for a given HPC system, and cannot be expected to learn how to estimate the associated resource usage accurately.

### 2.2. IO contention in HPC systems

IO contention is the result of aggregate jobs' IO demand exceeding the bandwidth of the IO pipeline between compute nodes and storage. A hardware solution to bandwidth limitations is not foreseeable in the next future: the growth in system compute power is outpacing the growth in system IO bandwidth. [Figure 1](#) shows this trend for three flagship HPC systems (i.e. Jaguar, Titian, and Summit) in the past decade. Consequently, applications may run faster, generate more data, and experience idle status while waiting to write the data to storage because of IO contention. We expect this chain of events to get worse as the gap further increases with the next generation of HPC systems [Sung et al. \(2019\)](#), [Neuwirth et al. \(2017\)](#).

Previous work has presented numerous ways that IO contention can manifest itself. In [McKenna et al. \(2016\)](#) and [Lofstead et al. \(2010\)](#), IO contention occurred during bursts of system IO activity, where IO demand temporarily exceeds IO capacity. In [Khetawat et al. \(2019\)](#), [Mubarak et al. \(2017, 2016\)](#), system network topology and IO nodes caused bottlenecks in the IO pipeline that lead to localized IO contention. In each case, IO contention is the result of co-scheduling IO-intensive jobs on the contended resources (e.g. the IO bandwidth). With IO-blind HPC schedulers, these sources of IO contention cannot be effectively identified, prevented, and mitigated. In [Di Martino et al. \(2014\)](#), IO contention and consequent degradation is due to the parallel file system (PFS) failure. PFS failures can be partial or total, and can occur in either the PFS software, the PFS hardware, or both. A combination of hardware and software techniques can help mitigate PFS failures, but in some cases these mitigation techniques can themselves become sources of performance degradation and failure. For example, while RAID protects against data loss after hard drive failures, it greatly decreases drive performance during parity rebuilds. Additionally, storage servers in a failover mode can automatically pass their drives to a neighboring server, but the process can sometimes hang or fail, leaving the drives of both servers unavailable. Aside from system



**Figure 1.** The gap between compute and IO capacity for three flagship machines in the past decade shows an increasing disparity between the capacity to generate data and the capacity to move the data to storage.

failures of the PFS, work in [Lockwood et al. \(2018\)](#) has showed how file system performance can go through long and short trends of stability and instability for a variety of non-trivial reasons. This means that the baseline performance of the PFS can fluctuate over time, independent of any detectable failure.

Because IO contention can have such far reaching consequences, affecting job runtime and thus decreasing job throughput and resource utilization, it is vital that it is addressed. Current HPC systems are ill-equipped to deal with the primary causes of IO contention, and new tools that can prevent and mitigate the negative effects of IO contention are necessary.

### 3. Our solution: The AI4IO tool suite

AI4IO is a software tool suite that addresses the problem of IO contention on HPC systems. The current implementation of AI4IO consists of two tools, PRIONN [Wyatt et al. \(2018\)](#) and CanarIO [Wyatt et al. \(2020\)](#). The combination of these two tools enables IO-awareness on HPC schedulers. Each tool augments the HPC scheduler with the knowledge necessary to enable IO-aware scheduling decisions. PRIONN and CanarIO approach IO-aware scheduling from orthogonal directions: PRIONN enables IO-aware scheduling by preventing the occurrence of IO contention, while CanarIO enables IO-aware scheduling by mitigating the negative effects of IO contention that does occur. The tools are complimentary in the IO-aware scheduling decisions

they enable. PRIONN can avoid the IO contention caused by IO bursts when many IO-intensive jobs are co-scheduled. CanarIO can mitigate the effects of increased job runtime from other sources of IO contention, such as performance degradation of the PFS. Together these tools can cooperatively address the problem of IO contention.

#### 3.1. PRIONN: Prevention of IO contention

PRIONN (Predicting Runtime and IO using Neural Networks) [Wyatt et al. \(2018\)](#) targets the prevention of IO contention on HPC systems. The PRIONN tool predicts runtime and IO for each job submitted, removing this responsibility from the user. The tool's workflow comprises three steps: (1) job scripts processing; (2) modeling runtime and IO with AI; and (3) training and predicting the per-job runtime and IO. These steps are performed asynchronously to the scheduling of jobs on a production cluster.

*Job scripts processing* transforms raw text from job scripts into data that can be input into AI models. Traditional machine learning models for resource predictions such as Random Forest (RF), Decision Tree (DT), or k-Nearest Neighbor (kNN) rely on manual parsing of and feature extraction from job scripts: specific features (lines) in job scripts are identified, parsed, and transformed into data useable by machine learning models [Smith et al. \(1999\)](#), [McKenna et al. \(2016\)](#), [Rodrigues et al. \(2016\)](#). A novel component of PRIONN is the automatic mapping of entire job scripts to an image-like data representation (i.e. a tensor-

based data representation of each job script) that can be input into deep learning models for predictions. By doing so, a manual parsing and interpretation of the job scripts' structure (e.g. type of lines) are no longer needed.

Each image-like representation of a job script is composed of pixels which are mapped from text characters in the job script. Job scripts are cropped and padded to a fixed size because deep learning models require a constant size input. We map each job script into a 2D matrix of pixels (i.e. numerical values). Each pixel is mapped from a character in the job script. Previous work by Wyatt et al. (2017, 2018) showed how this 2D matrix structure outperforms others representations.

Scientific workloads are increasingly incorporating ensemble simulations, in which single users or group of users inject up to millions of jobs into the HPC system, resulting in sets of job scripts with common patterns. When mapping job scripts to a 2D matrix we preserve the structure of the job script text, since the location of characters in the job script are not altered (i.e. each character is mapped to a pixel). Furthermore, our image-like representation of job scripts can fully and automatically detect repetitive features and patterns in ensembles of jobs belonging to the same users or group of users.

The mapping of characters to pixels is performed on a separate node from the batch scheduler to avoid interference with the critical path of the resource scheduler. The mapping can be performed with different transformations. In PRIONN, we use the *word2vec transformation* Mikolov et al. (2013) method that examines the context of a character (i.e. surrounding characters) to embed information about that character in a multidimensional vector. In Wyatt et al. (2017, 2018), we show how the *word2vec transformation* mapping more accurately allows us to embed information about each character in the 2D matrix than techniques such as binary mapping or simple one-hot encoding. Specifically, the word2vec transformation method is a lossless transformation that converts each unique character to a vector. We use Google's word2vec method to obtain this transformation. This method examines the context of a character (i.e. surrounding characters) to embed information about that character in a multidimensional vector. Motivated by the need to keep the data processing and training time low, we fix the job scripts to a standard size of 64 rows and 64 columns of characters before mapping the data to our image-like representation. Job scripts with less than 64 rows or columns of characters are extended to this size with space characters. Jobs larger than 64 rows or columns of characters are cropped to the correct size. Standardizing the size of job scripts incurs a small amount of data loss at the very end of the impacted scripts. Note that only 9.9% of jobs scripts contain more than 64 lines of text and 13.8% of text lines contain more than 64 characters. We can reason that word2vec compared to other methods (e.g. binary mapping

or simple one-hot encoding) because each character is encoded with information about its context. In other words, the character is not considered in isolation but in the context of the other neighbored characters. This information can decrease the epochs needed to converge on an accurate deep learning model.

By mapping characters into pixels, our data mapping method in PRIONN has three advantages over manual feature extraction used in previous resource prediction work such as Smith et al. (1998, 1999), Krishnaswamy et al. (2004), Cunha et al. (2017), Chen et al. (2013), McKenna et al. (2016), Rodrigues et al. (2016). First, PRIONN is automatic and can be deployed as-is on any HPC system. Second, our tool automatically scales with an increasing number of HPC jobs and HPC systems in that it does not require ongoing maintenance as job scripts and HPC environments change. Third, our tool is general to any HPC system. Any type of job script can be processed with our mapping to produce data which can be used with a deep learning model.

The *AI modeling* used by PRIONN is driven by the trade-off between prediction accuracy and prediction costs (i.e. training time). These properties are orthogonal: if a prediction method is highly accurate but slow to produce predictions, delays will make the prediction useless for the batch scheduler. We target our tool to be as accurate as possible without impacting the performance of the scheduler. In McKenna et al. (2016), Wyatt et al. (2017, 2018), we assessed a diverse set of AI models for the jobs' feature extractions, showing how a 2D Convolutional Neural Network (2D-CNN) model provides the better trade-off values. The 2D-CNN uses the 2D image-like data representations of job scripts as input and contains several 2D convolutional layers followed by several fully connected hidden layers. Specifically, PRIONN uses the word2vec algorithm with an output vector size of four and a 2D-CNN with four convolutional layers and four fully connected layers. Furthermore, the deep learning model is a classifier: each node in the final output layer is associated with a range of values predicting resource usage (e.g. for runtime predictions, the output layer is 960 nodes in size where each node is associated with a runtime in minutes between 0 and 960 min).

We perform *training and prediction* at the time of job submissions. For the training, historical jobs' data (i.e. jobs that have already executed) are used to train the 2D-CNN model. The submission, start, and end times of these jobs are used to replicate the queuing and execution of jobs on a HPC cluster with its SLURM batch scheduler. The job scripts are transformed into image-like representations. Because prediction of job runtime and IO resource should occur at the same time as the job submission to the batch scheduler, after every 100 job submissions, the 2D-CNN model is trained with the newest historical job data. We train

the 2D-CNN model on the 500 most recently completed jobs. Our empirical evaluation of training with data from 50 up to 5000 jobs indicated that there is only a minor improvement of prediction accuracy and higher cost to train beyond 500 jobs for PRIONN. The time necessary to process 500 job scripts (i.e. the number of jobs used each time we train the 2D-CNN model) ranges between 9 and 10 s. The training is performed offline without interfering with the scheduler and is repeated after 100 jobs have been scheduled by using the past 500 completed jobs as historical data. The trained model is then used to predict the resource usage (i.e. runtime and IO) of each job as it is submitted to the batch scheduler based on its job script. When evaluating prediction models, we first consider the accuracy of the predicted resource usage. Then, we consider the time needed to obtain the resource usage prediction. To this end, we compare predicted resource usage to the actual resource usage for each job with relative accuracy. Relative accuracy is preferred over absolute error because it mitigates the negative impact of small prediction error for jobs with high resource usage. For example, a runtime prediction error of 30 min is far worse for a one-h job than for a 12 h job. Equation (1) shows how we calculate relative accuracy for each resource usage prediction, where *true* is the actual value and *pred* is the predicted value. The  $\epsilon$  value in the denominator (i.e. machine epsilon) prevents division by zero when both *true* and *pred* are 0.

$$relativeAccuracy = 1 - \frac{|true - pred|}{\max(true, pred) + \epsilon} \quad (1)$$

We use the maximum value between *true* and *pred* in the denominator of equation (1) for two reasons: (1) to maintain a range of [0, 1] for the metric and (2) to penalize under-prediction more than over-prediction. We do this because under-predicted resource usage (e.g. we predict IO of 10 MB/s for a job that uses 25 MB/s) fed into an IO-aware scheduler could result in resource contention.

### 3.2. CanarIO: Mitigation of IO contention

CanarIO Wyatt et al. (2020) targets the mitigation of IO contention effects during job executions. While per-job predictions from PRIONN can be used to prevent IO contention caused by co-scheduling IO-intensive jobs, additional sources of contention such as PFS downtime persist. Current batch schedulers enable users to address this issue manually. Users can specify needed file systems in their job submission script, and facility staff can mark particular file systems as unavailable. In many cases, facilities can find out when major outages happen in the file system by looking at facility monitoring tools, but these tools do not help users understand which jobs can be run in an outage. CanarIO leverages fine-grained metric data from

the Lightweight Distributed Metric Service (LDMS) and “canary” jobs to monitor IO degradation and quantify the sensitivity of jobs to IO degradation. CanarIO’s name is derived from the canaries used by coal miners to detect toxic gas in mine tunnels: if a canary job is killed when it runs out of allocation (or has increased runtime), it can be an indicator of degraded performance for the entire system. The CanarIO’s workflow comprises three steps: (1) monitoring of periodic IO-probing jobs (i.e. canary jobs) to identify when IO performance degradation occurs; (2) annotating historical jobs that are either sensitive or resistant to the observed IO performance degradation; and (3) training of AI models to detect real-time IO degradation and predict which jobs are sensitive to that IO degradation.

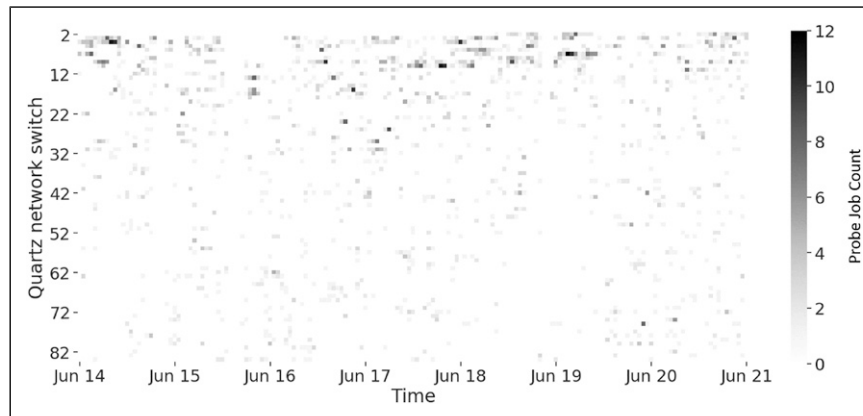
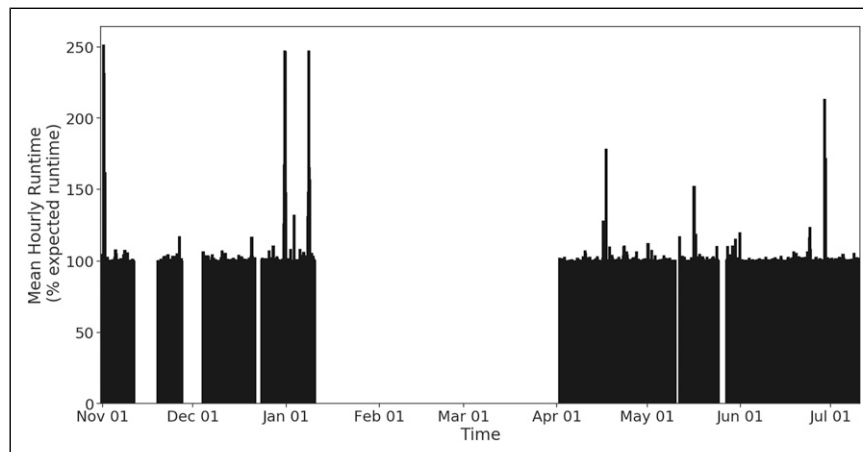
To monitor performance degradation, we use a set of Multi-purpose, Application-Centric, Scalable I/O Proxy Application (MACSio) jobs, run at regular intervals, to collect high-granularity IO-related performance data from the system Miller (2015). It is common practice to run benchmarking applications, such as MACSio, on HPC systems to collect information about the performance of system resources; however, these benchmarks are commonly run, at most, once per day. On the contrary, in CanarIO the MACSio jobs run continuously. We model five common IO patterns in five MACSio probes. Each probe is a single node, 36 process execution of MACSio. We adapt parameters for MACSio that affect the total IO, number of output files per process, and number of Lustre Object Storage Targets (OSTs). The five probes and these details are listed in Table 1.

Each probe is submitted to the system queue as a single-node job. We maintain five jobs for each probe in the system queue at all times to collect IO performance data from the system on a regular and frequent basis. We allow the system scheduler to randomly place our jobs across system nodes, ensuring more complete sampling of nodes and network switches on the system. Empirically, we collect data to investigate the cause of observed slowdowns in probe executions and observe that when a probe job slows down, it is exclusively due to file system IO performance. This assumption is based two observations: (i) the overhead for running our probe jobs is minimal for the HPC systems in our testing and (ii) none of the five MACSio probes are computationally, memory, or network intensive Wyatt et al. (2020).

We define any probe job as being performance degraded based on the comparison of observed probe job runtime and expected probe job runtime. In fact, each of our probes has an expected execution time identified with PRIONN. The expected runtime is calculated by taking the median runtime of the first 100 executions for each probe. We use a threshold runtime of 105% the expected runtime to define performance degraded jobs (i.e. jobs exceeding the threshold runtime are performance degraded). The low

**Table 1.** The five probe jobs that we run to detect IO degradation events on the Quartz system at LLNL model common IO access patterns.

| Benchmark | Total write IO (GB) | Files per process | Expected runtime (s) | Description           |
|-----------|---------------------|-------------------|----------------------|-----------------------|
| Probe-1   | 100                 | 10                | 1396                 | Few, large outputs    |
| Probe-2   | 100                 | 100               | 1023                 | Many, small outputs   |
| Probe-3   | 2                   | 100               | 519                  | Many, tiny outputs    |
| Probe-4   | 100                 | 10                | 521                  | 1, w/Max LSF striping |
| Probe-5   | 100                 | 100               | 1408                 | 2, w/Max LSF striping |

**Figure 2.** A heatmap showing the time of execution and network switch used for our probe jobs over an average week.**Figure 3.** Runtime of our “Canary” probe jobs over time. The spikes above 100% indicate a longer runtime than expected and the detection of a possible IO-related degradation event.

threshold (i.e. +5%) is supported by the low overhead for running our probe jobs. Observations over a total of 571 node-hours over 7 days during which we ran our probe jobs on Quartz at LLNL indicate that only 0.11% of node-hours available on the machine were used by our probe jobs. If we expand the time window to the entire 8 months of data collection, only 0.04% of all cluster node-hours were used to run CanarIO probes. In addition to using negligible

machine time, our probe jobs are often scheduled as back-fill jobs. Given their short runtime and capability to fit into small back-fill windows, they have limited impact on the scheduling and execution of other jobs on the machine.

We cross-validate the IO usage patterns of the CanarIO probes jobs with PFS log files to validate the IO causality and understand the IO problem at the root of IO slowdown. Figure 2 shows a heatmap indicating where and when our

probe jobs ran during a week of monitoring on the Quartz cluster. We achieve good coverage of the system topology and maintain a running probe job during most time intervals. Figure 3 shows the runtime of over 20,000 probe jobs, averaged by hour, over 8 months ranging from November 2018 to July 2019 on the same LLNL cluster. The figure shows small daily and hourly fluctuations in observed probe runtime with several large, isolated spikes that last several hours or days. Taller and wider spikes represent more severe performance degradation. The CanarIO probe jobs detect both small and large performance degradation events. The gaps in the data are due to the temporary downtime of our data collection pipeline.

For training CNNs, we need historic data of annotated jobs that are classified either IO-sensitive and IO-resistant jobs. To *annotate jobs in a generic application*, we cannot rely on the application codes or the input decks of the user-submitted jobs. In our solution, we first collect job's IO usage data from LDMS. The IO usage data is a total count of bytes sent or received by a node, including network activity. Lustre specific LDMS counters are implemented in LDMS v3. Note that this service is not supported by the Lustre system attached to most clusters, making the collection of IO data not always feasibly across HPC systems. Then, we define a method that uses a set of nested filters together with job metadata (i.e. data from our probe jobs and LDMS IO usage data) to complete the annotation. Specifically, the nested filters are used to annotate jobs as IO-sensitive and IO-resilient. A first filter selects only user-submitted jobs that have an execution time longer than 1 min and run multiple times. A second filter selects only jobs that overlap in execution times with one or more of our probe jobs. From those jobs, a third filter selects jobs with execution time that is 5% or larger than their expected runtime and marks them as IO-sensitive; the other jobs are marked as IO-resilient. In other words, the nested filters allow us to label executed jobs and provide us with the annotated dataset needed to train the predictive model components of CanarIO.

We empirically validate our method by applying it to a dataset of 767,999 jobs that ran between November 2018 and July 2019 on the Quartz cluster. For each job, we collect the job script text, user ID, job ID, job name, start time, end time, requested runtime, and node count. We identify and remove from our datasets all the jobs that terminate at their initialization. These jobs are identified by runtimes of less than 1 min. Additionally, we remove jobs that occupy the system for less than 30 node-minutes. We reason that these low resource jobs have negligible impact on the IO degradation we study. This results in a reduced-in-size dataset of 658,229 jobs. Using the job script, user ID, job name, and node count as the identifier of a job, we count a total of 126,310 unique jobs in the dataset. Following the same approach applied to our probe jobs, we calculate the expected runtime of jobs in the dataset. For each job, we

compare the observed runtime to the expected. Because we do not have the application code and input decks for the user-submitted jobs in our dataset, we must infer the expected runtime based on observations of jobs that are run multiple times. We remove jobs that were run only once from our dataset and analyze the remaining 18,538 unique and 390,726 total jobs. We calculate the expected runtime for each unique job as the minimum observed execution time.

Real user-submitted jobs may experience performance degradation for a number of non IO-related reasons. We reduce the probability of selecting these jobs with non IO-related performance degradation using information collected from the probe jobs. Because our probe jobs are only IO-intensive, we can deduce user-submitted jobs with IO-sensitivity (or resilience), if they are executed at the same time as our probe jobs. By considering only user-submitted jobs that overlap in execution times with one or more of our probe jobs, we further reduce our dataset to 7557 unique jobs and 69,582 total jobs.

We label our dataset of user-submitted jobs as IO-sensitive or IO-resistant based on the expected runtimes and information from our probe jobs. If the runtime of a job is 5% larger than the expected runtime, we label it as performance degraded. For each unique job that has at least one execution that demonstrated performance degradation, we assert that the performance degraded executions must overlap with a probe job that also exhibited performance degradation. These jobs are labeled as IO-sensitive. Similarly, we define IO-resistant jobs as jobs that show no performance degradation but also have one or more executions that overlapped with a performance degraded probe job. We identify a total of 6358 unique jobs and 47,969 total jobs that match our criteria for IO-sensitive and IO-resistant jobs.

As a final validation, we consider the IO usage data for each job in our dataset. We calculate the total read and write IO usage of each job from traces generated with Caliper Boehme et al. (2016) and Hatchet Brink et al. (2020) and compare these values for executions of each unique job. For IO-sensitive jobs, the total IO of expected runtime executions must be equal to or greater than the total IO of performance degraded executions. This ensures that runtimes greater than the expected runtime are not caused by parameter changes in job input decks that generated more IO activity. The resulting dataset contains 4906 unique IO-resistant jobs and 1293 IO-sensitive jobs.

To *train AI models* for predicting whether a job is IO-sensitive or IO-resistant, we use the real queue, start, and end times of jobs queuing and executing on the HPC cluster. Data from historical jobs (i.e. jobs that have already executed) are used to train the AI model. The trained model is then used to predict the IO-sensitivity of jobs as they are submitted to the cluster (i.e. before execution). We

incorporate the CNN from the PRIONN tool [Wyatt et al. \(2018\)](#) into CanarIO for the purpose of predicting IO-sensitive and IO-resistant jobs. This model is ideal for its ability to use all the information available in user-submitted job scripts to make accurate predictions about job resource usage. We maintain the CNN layer structure used for PRIONN and adapt the output layer of the model to make the binary prediction for a job being IO-sensitive or IO-resistant from only job scripts.

#### 4. IO-aware flux-based testing environment

We rely on the Flux production workload manager given by [Herbein et al. \(2016\)](#), [Ahn et al. \(2020\)](#) to study a HPC environment augmented with AI4IO. To this end, we extend the original Flux workload manager and its simulator to incorporate PRIONN and CanarIO. Each tool in the AI4IO suite enables IO-aware scheduling in a different way. By combining the two tools in Flux, we can measure the impact of PRIONN and CanarIO both in isolation and in concert. To reproduce and explain results, we study the impact of AI4IO across scenarios on the Quartz HPC system at LLNL Qua (2020) (a 36 core-per-node, Intel Xeon E5-2695 system with 3–18 nodes and 344,064 GB total memory) with the support of the Flux simulator. To this end, we extend the simulator, which was previously validated by [Herbein et al. \(2016\)](#), to mimic the effects of IO contention and user behavior when jobs are unexpectedly terminated. Under these circumstances, users tend to resubmit some of the prematurely terminated jobs.

##### 4.1. Flux scheduler and simulator

Flux introduced by [Herbein et al. \(2016\)](#), [Ahn et al. \(2020\)](#) is among a few of the already available next-generation workload management software systems supporting (a) HPC system configurations including a PFS mounted on multiple clusters and (b) a global view over all cluster resources from which jobs can access the mounted PFS. Flux responds to the need for scheduling beyond system boundaries. Through its simulator, Flux presents an opportunity for testing our methodology in a controlled environment that assures reproducible results.

Flux uses a generalized resource model, known as the Resource Description Language (RDL), that allows the modeling of arbitrary resources and the hierarchical relationships between them ([Ahn et al., \(2020\)](#)). Flux's RDL is implemented as a Lua library with a C interface, which provides interoperability with Flux's scheduler. Within Flux's RDL, each resource type is represented by a Lua class. In previous work, we extended Flux to be IO-aware: we defined new Lua classes representing the resource types

within our target I/O subsystem: gateway nodes, network switches, and PFS. Using these resource types within Flux's RDL, Flux models the IO subsystem as a hierarchy of network switches and gateway nodes leading to a PFS and establishes the hierarchical relationships between the resources. Each of these new resource classes contains an *IO limit* attribute, which represents either the IO bandwidth capacity of the resource itself (e.g. the PFS can read and write at a maximum of 1 GB/s) or the bandwidth limit of the resource's connection to its parent (e.g. a compute node can communicate with its closest network switch with a maximum bandwidth of 128 MB/s) ([Ahn et al., \(2020\)](#)).

The Flux simulator allows us to test job executions without having to launch real jobs. Users submit jobs to the Flux scheduler that launches them based on a defined scheduling policy. The Flux simulator entirely removes any user interaction and replaces it with an auto-submission module. Moreover, the simulator does not rely on the direct execution of jobs but on the simulation of their execution. It can use the Slurm database of submitted jobs to generate the profile of simulated job executions. As real-time is too slow for the testing, the simulator triggers events synthetically. By using the simulator, we can rapidly study critical questions associated with the job scheduling and resource management.

##### 4.2. Flux simulator's new features

We use the existing Flux simulator as a starting point for building a realistic HPC environment and evaluating AI4IO-enabled IO-aware scheduling. To this end, we extend the simulator to mimic IO contention and enable IO-aware scheduling. These extensions to the Flux simulator provide the means for comparing traditional scheduling with scheduling augmented with AI4IO tools on a given HPC system.

The available Flux simulator is capable of simulating the scheduling and execution of jobs from input data containing jobs' resource usage (i.e. node count and runtime), submission time, and time limit. The simulator creates a virtual HPC system with a set of homogeneous nodes and an instance of the Flux scheduler that provides an interface to the virtual systems. To support the study of AI4IO-augmented schedulers, we extend the simulator in two ways: (1) we add PFS IO as a simulated resource in order to develop an IO contention model that affects job execution; and (2) we introduce mechanisms that mimic user response to increased job runtime.

Our first extension to the Flux simulator enables the simulator to track PFS IO and uses a model to generate IO contention. We include the average IO bandwidth usage during execution for each job and track the aggregate IO bandwidth demand of running jobs. We assume that all IO on the system is directed to an attached PFS. Therefore, we

implement a limit to the IO bandwidth available to the entire system to mimic the maximum IO bandwidth we observe on the HPC system and its associated PFSs. When the IO limit of the system is less than the aggregate IO demand of executing jobs, IO contention occurs. IO contention is implemented in the Flux simulator such that the runtime of an IO-sensitive job increases while the runtime of an IO-resistant remain unchanged. This contention model is based on previous contention modeling work in Dietze et al. (2018), Mohammed et al. (2019), Martinasso and Méhaut (2011). Greater values of system IO demand above the IO limit creates more severe IO contention. Contention ends when the IO demand drops below the system's IO limit. Additionally, IO contention events can be directly injected into the Flux simulator. This allows us to simulate other sources of IO contention such as partial outages of PFS resources. During the simulation, we inject IO contention modeled from IO contention events we observed in data collection from LLNL's HPC systems Wyatt et al. (2020).

The second extension to the Flux simulator adds mechanisms to mimic user response to IO contention's effects on jobs. Specifically, we emulate the action of users to rerun their jobs when they are canceled for exceeding the time limit. When IO contention extends job runtime, there will be jobs for which the contention causes the runtime to exceed the job's allocated time limit. As a result, the entire job run or part of the job run may be discharged and the job's results may be lost. Users tend to resubmit these jobs to complete their work. We observe this user behavior in historical data collected for PRIONN and CanarIO. Accordingly, in the extended simulation, when a job exceeds its time limit, it is resubmitted with a longer time limit (i.e. double the original requested time).

#### 4.3. IO-aware scheduling

PRIONN predictions consist of the individual jobs' runtime and IO bandwidth, supporting the scheduler in determining if a job should start or should be delayed. CanarIO predictions classify jobs into IO-sensitive and IO-resistant, supporting the scheduler in determining if an IO-sensitive job should be replaced with IO-resistant jobs. AI4IO predictions are a combination of PRIONN and CanarIO predictions.

We implement an "AI4IO Oracle" that generates PRIONN and CanarIO predictions from a given prediction accuracy. The Oracle predictions use the real job values (e.g. IO and runtime) and given prediction accuracy to generate the PRIONN and CanarIO predictions. Predictions are generated from a normal distribution of accuracy values centered at the given accuracy (i.e. mean accuracy). The spread of these distributions is determined by fitting a normal curve to the PRIONN and CanarIO prediction distributions of real HPC data. We use two settings for the

Oracle's prediction accuracy. First, we model the prediction accuracy from those measured on real datasets; second, we test 100% accurate predictions.

The use of the Oracle is beneficial for two reasons. First, users may not have all of the necessary inputs from the simulation to properly train PRIONN and CanarIO (e.g. LDMS IO usage). Second, an Oracle allows users to precisely test a wide range of AI4IO prediction accuracy repeatedly over many datasets.

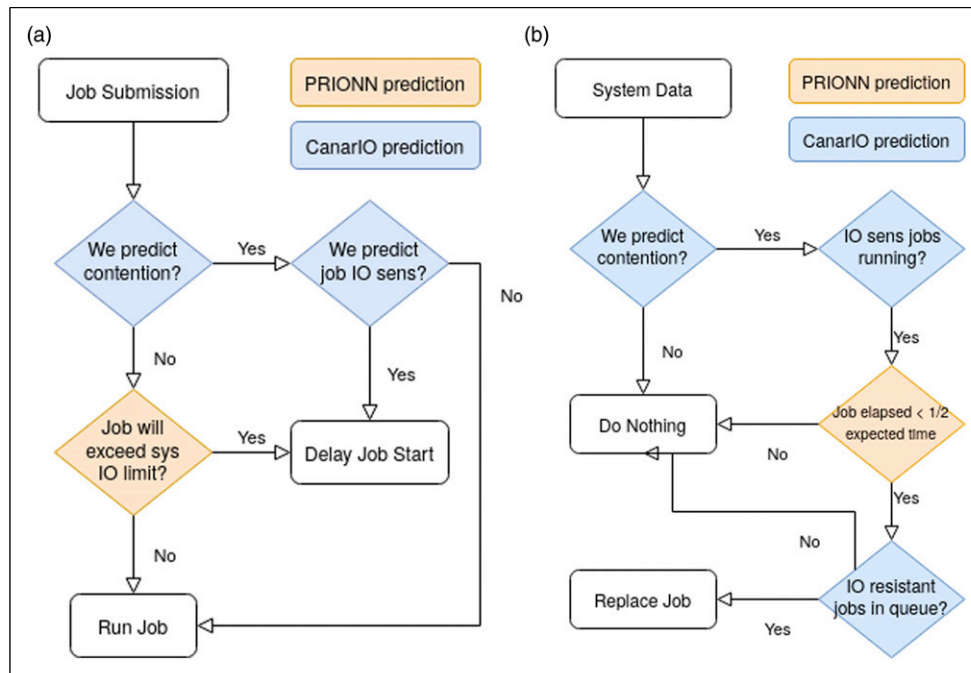
To enable IO-aware scheduling, we develop a decision logic that allows us to transform AI4IO predictions into job scheduling actions capable of preventing and mitigating IO contention. Figure 4 demonstrates the two pieces of integrated logic that generate IO-aware scheduling from PRIONN and CanarIO predictions. The added logic depicted in Figure 4(a) shows whether a job should start or should be delayed. We first query CanarIO for information about IO contention on the HPC system and the IO-sensitivity of the job at the top of the queue. If we predict that an IO-sensitive job can be negatively impacted by IO contention, we delay the start, mitigating the impact of contention. Furthermore, we check whether the predicted IO usage of the job can cause the system's IO to exceed the IO limit. If it is the case, we prevent contention by delaying the job's start. When a job is delayed, it allows other jobs in the queue to be considered for running. Figure 4(b) shows the logic implemented to take action when there is IO contention and IO-sensitive jobs should be replaced by IO-resistant jobs. If IO-sensitive jobs have run for less than half their expected runtime and there are available IO-resistant jobs, we replace the IO-sensitive jobs with IO-resistant jobs to mitigate the impact of IO contention on the overall job throughput.

#### 4.4. IO-aware scheduling evaluation

We validate the capability of the Flux scheduler to react to our IO-aware decision logic at a small scale by qualitatively and quantitatively studying four variations of a real scenario in which we progressively add IO contentions, user response to jobs exceeding their time limits, and AI4IO-driven prevention and mitigation strategies. We measure the performance gains of Flux with and without AI4IO on a larger scale production system that mimics HPC systems at LLNL. All our tests are performed using the Flux simulator.

#### 4.5. Small-scale validation of flux using AI4IO

We empirically validate Flux using AI4IO. To this end, we simulate the behavior of a HPC system with 100 homogeneous 16-core nodes similar to clusters at LLNL. We assume that the cluster is attached to a shared PFS. Historical data for the training of the PRIONN and CanarIO models are from traces of Quartz (2020). For each test, we



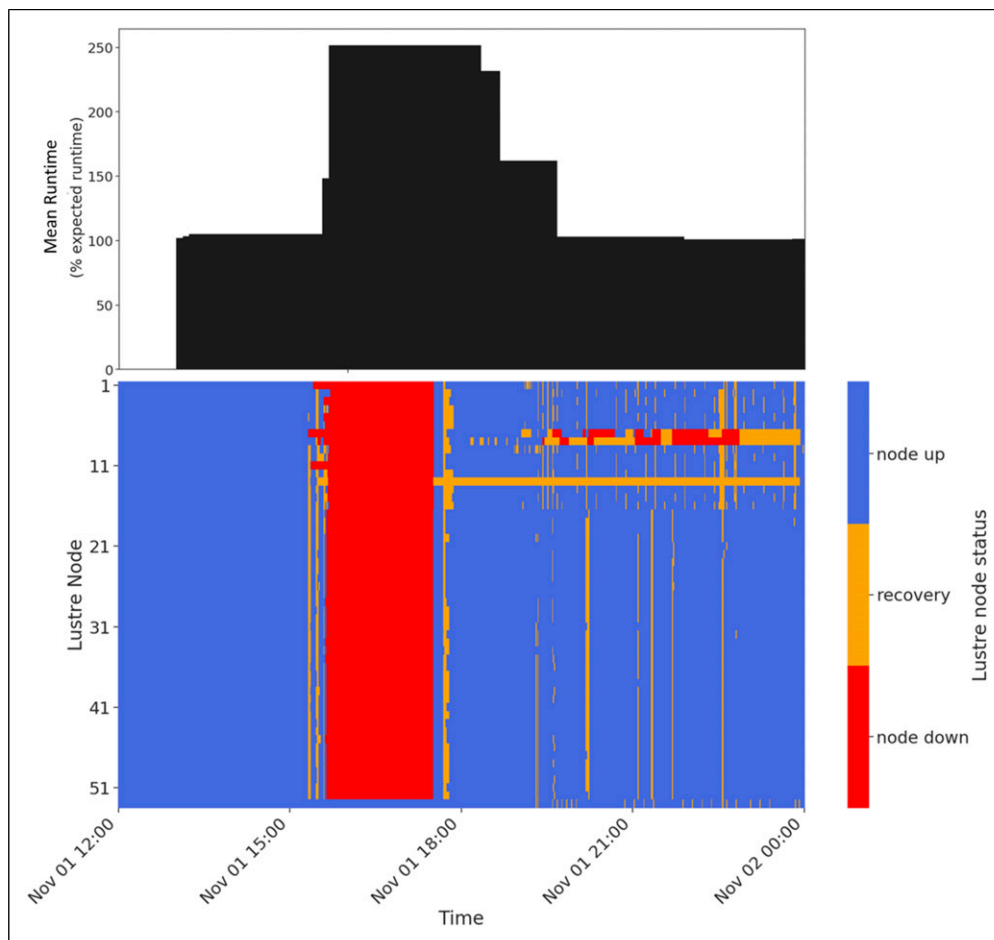
**Figure 4.** Rules supporting the decision logic implemented in the Flux simulator to integrate PRIONN and CanarIO predictions for IO-Aware scheduling. In (a) we show how to decide if a job should start or should be delayed; in (b) we show how to decide when to replace IO-sensitive jobs with IO-resistant jobs if IO contention occurs.

sample 10 sets of 1000-job workloads from the Quartz traces. For each set, the job submission times are spread across a 12-h window; the window is fed to the Flux simulation before each test run. Jobs that exceed their allocated times because of IO contention are rerun with 50% probability. We inject two types of IO contention events: a set of IO contention events associated with excessive IO demand that cause IO-sensitive jobs to progress 10–20% slower and single IO contention events associated with a major PFS crash that slows IO-sensitive job progress by 80–90%. These contention events are modeled from the contention events observed on Quartz. Figure 5 shows an example of these contention events observed on 1 November 2018. In the top plot of Figure 5, we observe a large spike in the execution time of our probe jobs (used by CanarIO) that occurs suddenly and subsides several hours later. In the bottom plot, we observe a correlated crash of nearly every node on the Lustre PFS at the same time as the probe jobs’ spike occurs. After the crash, many nodes go into an extended period of a recovery phase and some Lustre nodes crash again. During this time, we also see that some probe jobs have degraded performance in the range of 20–30% while other jobs have the expected runtime. Studies of this and other PFS crash use cases are presented in Wyatt et al. (2017) and indicate that Lustre logs alone are not reliable for determining IO performance degradation.

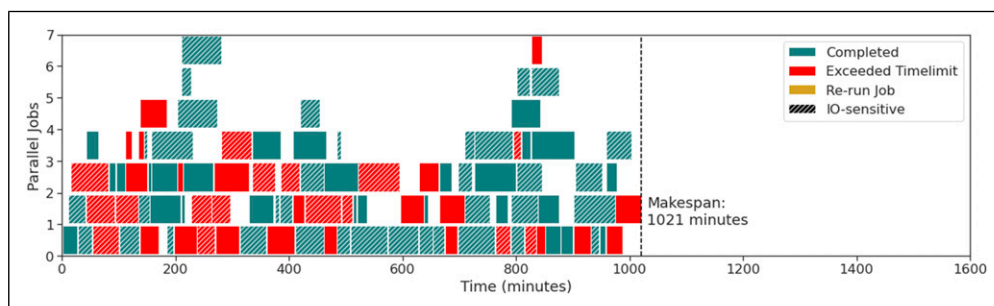
To empirically validate the HPC environment with these parameter settings and our extension to the Flux simulator,

we run and visualize multiple smaller-scale scenarios of a 100-job sampled workload on a Quartz-like 16-node cluster. In a first scenario, we show the base Flux simulation (i.e. Flux without AI4IO, without considering PFS IO, and with no IO contention). Figure 6 shows the execution of the 100-job workload. In the figure, each job is represented by a rectangle where the width of the rectangle indicates the job execution time. We use colors to annotate important information about the jobs: teal is for jobs with normal execution time, and red is for jobs that exceed the assigned time limit and thus are terminated before their completion. While those IO-sensitive jobs are already marked with a diagonal stripe pattern, the Flux simulator cannot and does not differentiate between IO-sensitive and IO-resistant jobs without our AI4IO augmentation. Furthermore, because Flux does not include any of the AI4IO features, no job is early terminated and rerun. The total makespan of the entire workload is 1,021 min, with 63 of the 100 jobs successfully completed over the 12-h interval of time. This workload makespan is considered the ideal lower bound for the workload.

In a second scenario, we introduce IO contention to the Flux simulator. Figure 7 shows the execution of the same set of 100 jobs and the system’s IO bandwidth demand. Behind the rectangles representing jobs, we collocate pale-red rectangles that show the occurrence of IO contention on the system. The first IO contention event in Figure 7(a) is labeled “PFS perf. degradation” and coincides with the



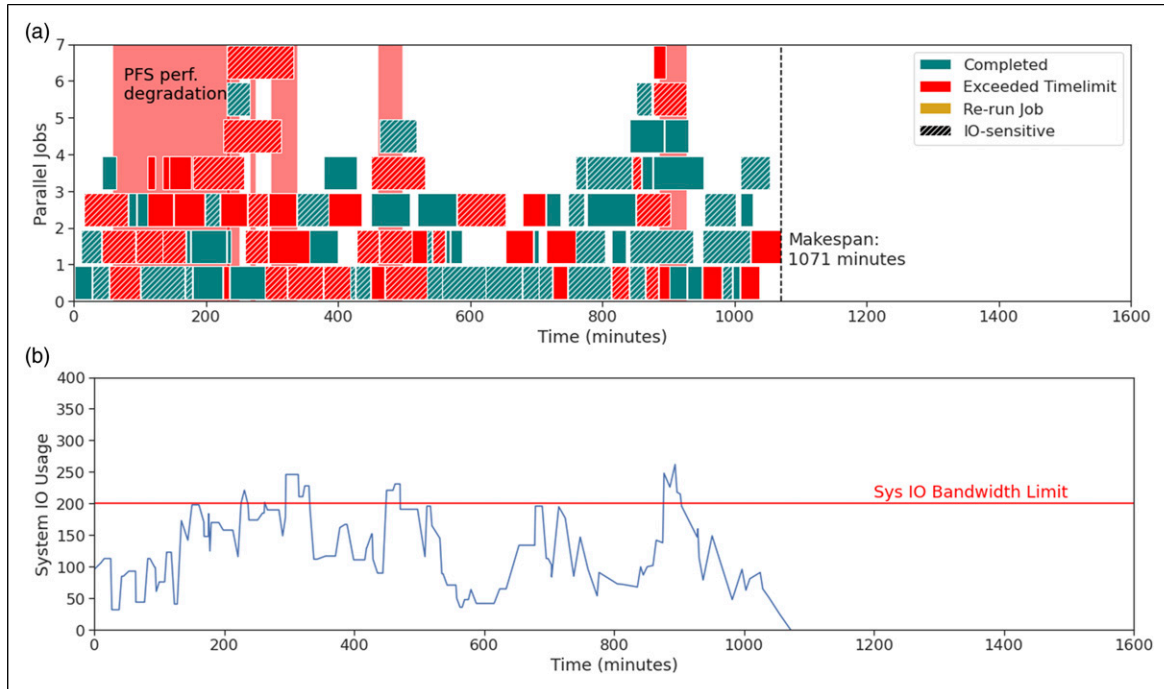
**Figure 5.** Example of contention event on Quartz with the observed runtimes of probe jobs (top) and Lustre node status (bottom) for a 12-h window starting on 1 November 2018 and showing the correlation between performance degradation of jobs and a Lustre PFS crash.



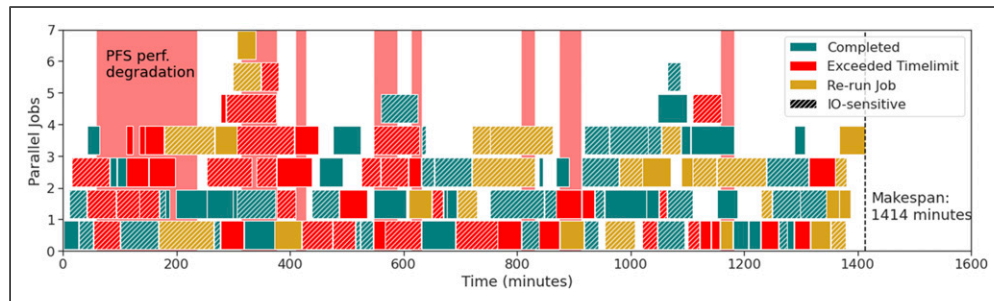
**Figure 6.** Execution of a sampled 100-job workload on a 16-node cluster, simulated with the standard Flux simulator before adding the new features (i.e. PFS IO and IO contention) and augmenting it with AI4IO. The execution gives the ideal lower bound of 1021 min for the makespan of the targeted workload.

injection of a major IO contention event such as a Luster PDF crash. The other two occurrences of IO contention are caused by system IO demand exceeding the IO limit, as depicted in Figure 7(b) with the system's IO bandwidth limit. The presence of IO contention causes IO-sensitive

jobs (jobs marked with a diagonal stripe pattern) to increase in runtime; many more jobs now exceed their time limit, increasing the total makespan of the workload to 1,071 min, with 55 of the 100 jobs successfully completing over this longer interval of time. Note how the lower number of



**Figure 7.** Execution of the 100-job workload on a 16-node cluster, simulated with the standard Flux simulator after adding IO contention but before augmenting the simulator with AI4IO. The makespan of the targeted workload increases to 1071 min and more IO-sensitive jobs time out. Pale-red blocks in (a) represent IO contention occurrences caused by injecting a PFS crash and jobs' IO demand; the line in (b) represents the pattern for the jobs' IO demand over time.



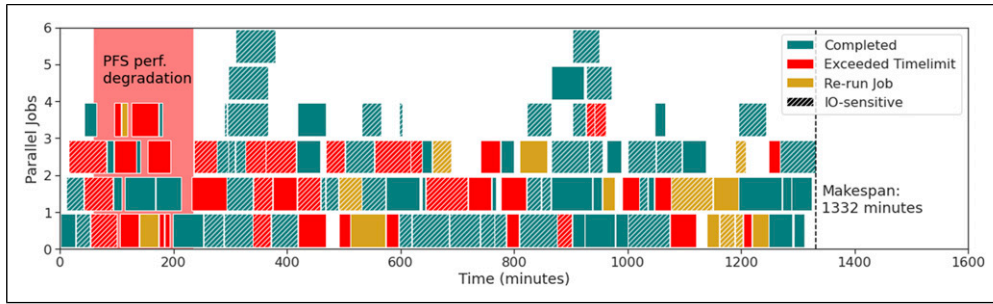
**Figure 8.** Execution of the 100-job workload on a 16-node cluster, the Flux simulator supporting IO contention and user response to jobs' early termination because exceeding their runtime. The makespan of the targeted workload substantially increases to 1414 min while more timed-out IO-sensitive jobs are resubmitted and rerun to mimic observed user behavior.

successfully completed jobs coincides with an increase number of IO-sensitive jobs that time out (i.e. 28 in Figure 7(a) versus 36 in Figure 6).

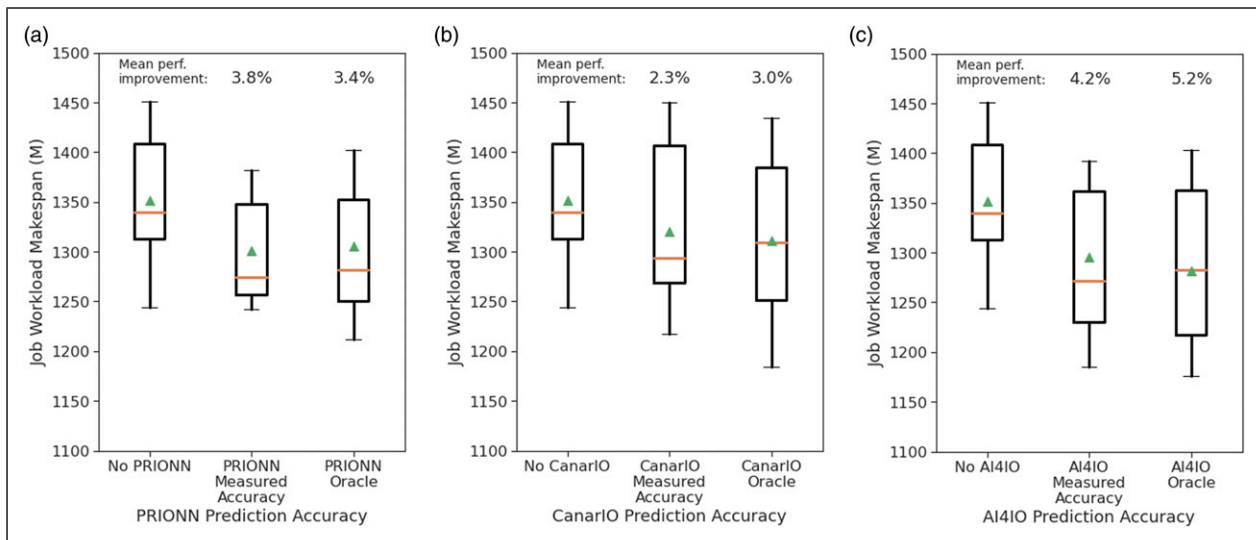
In a third scenario, we add the user response to jobs exceeding their time limits. In Figure 8, 50% of the jobs that exceed their runtime are resubmitted to the queue. These jobs are marked in yellow. We differentiate between resubmitted IO-resistant jobs from IO-sensitive jobs by using the diagonal stripe pattern for IO-sensitive jobs. We observe that IO contention still causes more IO-sensitive jobs to exceed time limits; by rerunning some of these jobs we further extend the makespan to 1,414 min. By including IO

contention and user behavior in this scenario, we have a more realistic execution environment than the previous two scenarios, but we are not yet using the AI4IO tools for prevention and mitigation. Consequently, Figure 8 shows that unchecked IO contention can inflate the makespan of a workload by more than 38%, to execute 68 of the 100 jobs in the targeted workload.

Our final scenario uses AI4IO tools to build on previous scenarios. We evaluate how the IO-aware scheduling works to prevent IO contention and mitigate IO impact. Figure 9 shows the same 100-job workload with IO contention, user response to timed-out jobs, and IO-aware scheduling with



**Figure 9.** Execution of the 100-job workload on a 16-node cluster with the Flux simulator supporting IO contention, user response to timed-out jobs, and IO-aware scheduling by using our AI4IO tools. The IO-aware scheduling uses predictions from PRIONN and CanarIO to prevent IO contention and mitigate IO effects on IO-sensitive jobs, resulting in a 1332 min workload and 78 of the 100 jobs completed.



**Figure 10.** Performance gain obtained by comparing IO-aware scheduling with IO-unaware scheduling decisions: in (a), we measure the impact of only PRIONN-based scheduling decisions; in (b), we measure the impact of only CanarIO-based scheduling decisions; and in (c), we combine PRIONN and CanarIO their combined impact on workload performance.

100% accurate predictions from our AI4IO Oracle. We observe the IO-aware scheduling is working in two ways: first, IO contention is prevented by keeping system IO demand below the IO limit, and second, during the contention caused by the PFS crash, less IO-sensitive jobs are run to mitigate the impact of the crash (as observed in Figure 5). As a result, the makespan of this workload shrinks by 6% to 1,332 min while increasing the number of job successfully completed to 78 of the 100 in the workload.

#### 4.6. Large-scale testing of flux using AI4IO

We use the augmented Flux simulator to evaluate the effectiveness of IO-aware scheduling enabled by AI4IO on a production cluster. To this end, we compare the makespan of 10 1000-job workloads of real HPC jobs simulated on a

large-scale Quartz-like, 100-node cluster with and without IO-aware scheduling. We test IO-aware scheduling with 100% accurate predictions and prediction accuracy matching those of PRIONN and CanarIO.

Figure 10 shows the results of our 10 simulated workload executions with and without IO-aware scheduling for three scenarios using the AI4IO tools. First, Figure 10(a) shows the distribution of makespans for the 10 workloads without PRIONN, with empirical PRIONN-based predictions preventing IO contention, and with 100%-accurate predictions (Oracle). We observe that just by enabling the PRIONN-based scheduling decision logic in Figure 4, we can reduce the mean makespan by >3% compared to an IO-unaware scheduler. Second, Figure 10(b) shows the distribution of makespans for the 10 workloads without CanarIO, with CanarIO-measured accuracy predictions mitigating IO

contention, and with 100% accuracy predictions (Oracle). We observe that enabling the CanarIO-based IO-aware scheduling logic in Figure 4 reduces the mean makespan by % compared to the IO-unaware scheduler. Last, Figure 10(c) shows the distribution of makespans for the 10 workloads without AI4IO (i.e. PRIONN and CanarIO), with AI4IO combining PRIONN-measured accuracy predictions preventing IO contention, with CanarIO-measured accuracy predictions mitigating IO contention, and with 100% accurate AI4IO predictions (Oracle). We observe that enabling IO-aware scheduling with both our AI4IO tools improves the mean makespan by 4.2% with our experimentally observed accuracy and by >5% with 100% accurate predictions (Oracle). The maximum makespan improvement for our experimentally measured AI4IO prediction accuracy is 6.4%. Figure 10(c) also shows that the combination of preventing and mitigating IO contention through PRIONN and CanarIO gives the greatest reduction in makespan.

While a reduction in makespan of 4.2% may seem like a small improvement on a cluster over a short interval of time, such a gain is significant if projected over long periods. If we consider a large cluster such as the entire 2600-node Quartz, AI4IO allows us to recover and reallocate more than 18,000 node-hours per week of operation that would otherwise be wasted.

## 5. Related work and novelty of AI4IO

The novelty of the proposed work is discussed in terms of its contributions to existing efforts to address resource contention and HPC job predictions.

### 5.1. Efforts to address resource contention

Many efforts have been made to quantify and address shared resource contention on HPC systems. Work such as Mubarak et al. (2016) and Martinasso and Méhaut (2011), has studied, modeled, and simulated network performance and contention in HPC. Mohammad et al. (2019) take a similar approach where they extensively model and simulate components of HPC systems, including network contention. Similarly, Dietze, Hofmann, and Runger analyzed and modeled contention to improve runtime estimates in Dietze et al. (2018).

In addition to the work toward understanding and modeling contention, solutions have been proposed which generally fall into two broad categories: application-centric and resource-centric. For example, an application-centric solution adjusts applications to match the available resources. This is evident in work from Dorier et al. (2014), which focuses on application coordination to avoid resource contention. Similarly, coordination and load balancing of application IO is explored in Neuwirth et al. (2016). Blagodurov and Fedorova (2013) also explore the use of

virtualization software to allow jobs to be migrated across a system when resource contention occurs. In contrast to the application-centric solutions, resource-centric solutions work to modify the resources themselves to avoid contention. For example, Neuwirth et al. apply load balancing of IO across the PFS storage nodes in Neuwirth et al. (2017). Similarly, in Xu et al. (2012), Zhao and Xu (2019), the virtualization of PFS resources for the purpose of load balancing is explored. The virtualization of HPC nodes themselves to accommodate resource-aware scheduling was also explored in Blagodurov and Fedorova (2012).

While many of these proposed solutions have been shown to benefit resource utilization, few focus on the problem of IO contention. Further, these solutions often involve extensive modification to the HPC environment. For example, in Blagodurov and Fedorova (2013), applications must be run with virtualization to enable the migration of jobs to address issues of contention. Because these solutions cannot be easily integrated with today's batch schedulers, their chance of broad community adoption is small. We address these shortcomings in this work by focusing on IO-aware scheduling that is easily integrated with popular batch schedulers like Slurm and automate the decisions with our AI-based tools.

### 5.2. Efforts to address HPC job prediction

The need for accurate job information is evident when considering how an IO-aware scheduler could address contention. The scheduler requires information about jobs, such as resource usage and IO-sensitivity, to adjust the schedule so that IO contention is prevented and/or mitigated. In this section, we discuss the extensive work that has been done to provide these predictions in the HPC environment and demonstrate how our AI4IO tools provide unique benefits.

In previous work, traditional machine learning models (e.g. Decision Tree and k-Nearest Neighbors) were primarily used to predict runtime Smith et al. (1998, 1999), Krishnaswamy et al. (2004), Cunha et al. (2017), Tsafir et al. (2007) and, in some cases, more general resource usage McKenna et al. (2016), Rodrigues et al. (2016), Matsunaga and Fortes (2010). This previous work has shown that machine learning models can provide more accurate runtime estimates than users Smith et al. (1999).

Smith, Foster, and Taylor use historical HPC job data to predict runtime in Smith et al. (1998, 1999). For each job runtime prediction, they train a linear regression model based on historically similar jobs. Job similarity is calculated with several extracted features, including user and job queue. Other runtime prediction papers use similar methods of manually extracted features and machine learning models as Smith, Foster, and Taylor. For example, Krishnaswamy, Loke, and Zaslavsky develop similarity templates for

predicting job runtime based on extracted features in [Krishnaswamy et al. \(2004\)](#). Cunha et al. use a kNN model to predict runtime and turnaround time for HPC jobs in [Cunha et al. \(2017\)](#). This work utilizes extracted features from job scripts augmented with data from the scheduler, such as the number of jobs in the queue at job submission time. Chen, Lu, and Pattabiraman present a method for predicting runtimes of jobs being executed using features extracted from log files and a hidden markov model [Chen et al. \(2013\)](#). Tsafir, Etsion, and Feitelson predict job runtimes based on a user-centric model in [Tsafir et al. \(2007\)](#). They use the average users' previous job runtimes as an estimation for the runtime of the next job submitted by a user. Downey develops a statistical model for predicting the queue time of a job based on jobs already running on an HPC system in [Downey \(1997\)](#). Similarly, in the work of Nurmi, Brevik, and Wolksi a statistical method called QBETS is developed to predict wait times for jobs [Nurmi et al. \(2007\)](#). Both of these works make predictions based on wait times of jobs currently running on a system and do not build a prediction model based on extracted features.

A smaller body of work has also focused on other job resources, such as IO. Lofstead et al. (2010) outline challenges of dealing with IO contention of parallel file systems and the benefits for preventing contention that come with knowing job IO behavior. Other works present methods for using extracted features to predict IO among other resources to prevent resource contention. Rodrigues et al. predict job runtime, wait time, and memory usage with an ensemble of machine learning algorithms, including kNN and RF, in [Rodrigues et al. \(2016\)](#). Their method extracts features from log files and batch scheduler logs. Matsunaga and Fortes (2010) investigate the prediction of many job features, such as CPU, memory, and IO usage with several machine learning algorithms using extracted features in

A common characteristic of this previous work is the requirement to develop and maintain parsers to extract features from a diverse source of data such as job scripts. When relying on these parsing techniques, these previous efforts have failed to generalize (i.e. different types of scripts require different parsers). Moreover, any efforts to create generic parsers incurs the cost of truncating and removing unique information present only in subsets of job scripts. In other words, not all information from job scripts can be captured by a parser.

### 5.3. Previous efforts of the authors

AI4IO addresses the shortcomings of previous work and builds on three blocks of the authors' previous work. First, in [McKenna et al. \(2016\)](#) we tested several traditional machine learning methods for predicting runtime and IO usage of HPC jobs. We tested kNN, DT, and RF models with manually extracted features from job scripts and job

logs, demonstrating the limits of manual IO data processing. Second, in [Wyatt et al. \(2018\)](#), we moved away from manual processing and, with PRIONN, we focused on automatic, general, and scalable AI methods for predicting accurate resource usage (i.e. runtime and IO usage). Last, in [Wyatt et al. \(2020\)](#), we focused on canary jobs (thus CanarIO) and their automatic application for training AI models capable to quantify IO contention and IO-sensitive jobs at runtime.

The work in this paper leverages the previous lessons learned and combines PRIONN and CanarIO into the cohesive suite of tools that is A4IO. We demonstrate that, both at the small and large scales, only by using PRIONN and CanarIO in concert, users can benefit of the best of both worlds (i.e. prevention and mitigation of IO contentions).

## 6. Conclusion

AI4IO, a suite of AI-based tools, enables IO-awareness in HPC workflow managers and their schedulers. The suite consists of PRIONN, which predicts runtime and IO usage of jobs before they execute, and CanarIO, which predicts job IO-sensitivity and detects in real-time the occurrence of IO contention. The tools can be integrated with existing workflow managers. In this paper, we augment the Flux workflow managers and its simulator with AI4IO. To this end, we develop IO-aware decision logic and integrate it into the Flux scheduler. We test IO-aware scheduling with PRIONN and CanarIO separately and in concert to demonstrate that the two tools enable prevention and mitigation of IO contention, respectively. While working in concert to prevent and mitigate IO contention, the two tools offer the largest improvement in resource utilization and performance. The makespan of real HPC job workloads increases in performance by an average of 4.2% with a maximum performance increase of 6.4%. At the same time, the job throughput of a realistic system experiencing bursts of IO demand from concurrently running jobs and PFS performance degradation, increases to match the throughput of an ideal systems without IO contention. On a production HPC system, the IO mitigation from AI4IO can result in more than 18,000 node-hours per week of operation that would otherwise be wasted.

### Declaration of conflicting interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

### Funding

The author(s) disclosed receipt of the following financial support for the research, authorship, and/or publication of this article: This work was supported by the National Science Foundation under Grants No. #1841758 and #1900888 and Department of Energy Grant

No. DE-AC52-07NA27344 (LLNL-JRNL-817993-DRAFT). Support from Lawrence Livermore National Laboratory is acknowledged.

## ORCID iDs

Stephen Herbein  <https://orcid.org/0000-0003-0141-0653>

Michela Taufer  <https://orcid.org/0000-0002-0031-6377>

## Supplementary Material

The source code is available in <https://github.com/flux-framework/flux-core/pull/2561>.

## References

- Quartz (2020) Quartz supercomputer: characteristics. Available at: <https://hpc.llnl.gov/hardware/platforms/quartz> (accessed 23 December 2020).
- Ahn DH, Bass N, Chu A, et al. (2020) Flux: overcoming scheduling challenges for exascale workflows. *Future Generation Computer Systems* 110: 202–213. DOI: [10.1016/j.future.2020.04.006](https://doi.org/10.1016/j.future.2020.04.006).
- Blagodurov S and Fedorova A (2012) Towards the contention aware scheduling in HPC cluster environment. *Journal of Physics: Conference Series* 385: 012010.
- Blagodurov S and Fedorova A (2013) Optimizing shared resource contention in HPC clusters. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC), Denver, CO, 22 November 2013, pp. 1–25.
- Boehme D, Gamblin T, Beckingsale D, et al. (2016) Caliper: performance introspection for HPC software stacks. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC). Salt Lake City, UT, 13–18 November 2016.
- Brink S, Lumsden I, Scully-Allison C, et al. (2020) Usability and performance improvements in Hatchet. In: Proceedings of the IEEE/ACM International Workshop on HPC User Support Tools and Workshop on Programming and Performance Visualization Tools (HUST/ProTools@SC), Atlanta, GA, pp. 49–58. Washington, DC: IEEE Computer Society. DOI: [10.1109/HUSTProTools51951.2020.00013](https://doi.org/10.1109/HUSTProTools51951.2020.00013).
- Chen X, Lu CD and Pattabiraman K (2013) Predicting job completion times using system logs in supercomputing clusters. In: Proceedings of the 43rd Annual IEEE/IFIP Conference on Dependable Systems and Networks Workshop (DSN-W), Valencia, Spain, 29 June–2 July 2020, pp. 1–8.
- Cunha RLF, Rodrigues ER, Tizzei LP, et al. (2017) Job placement advisor based on turnaround predictions for HPC hybrid clouds. *Future Generation Computer Systems* 67: 35–46.
- Di Martino C, Kalbarczyk Z, Iyer RK, et al. (2014) Lessons learned from the analysis of system failures at petascale: the case of blue waters. In: Proceedings of the 2014 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, Atlanta, GA, 23–26 June 2014, pp. 610–621. Washington, DC: IEEE.
- Dietze R, Hofmann M and Runger G (2018) Analysis and modeling of resource contention effects based on benchmark applications. In: Proceedings of the 2018 International Conference on High Performance Computing & Simulation (HPCS), Orleans, France, 16–20 July 2018, pp. 330–337. Washington, DC: IEEE.
- Dorier M, Antoniu G, Ross R, et al. (2014) CALCioM: mitigating I/O interference in HPC systems through cross-application coordination. In: Proceedings of the 2014 IEEE 28th International Parallel and Distributed Processing Symposium, Phoenix, AZ, 19–23 May 2019, pp. 155–164. Washington, DC: IEEE.
- Downey AB (1997) Using queue time predictions for processor allocation. In: Proceedings of the Workshop on Job Scheduling Strategies for Parallel Processing, Geneva, Switzerland, pp. 35–57.
- Herbein S, Ahn DH, Lipari D, et al. (2016) Scalable I/O-aware job scheduling for burst buffer enabled HPC clusters. In: Proceedings of the 25th ACM International Symposium on High-Performance Parallel and Distributed Computing (HPDC), Kyoto, Japan, 31 May 2016, pp. 69–80. New York, NY: ACM. DOI: [10.1145/2907294.2907316](https://doi.org/10.1145/2907294.2907316).
- Khetawat H, Zimmer C, Mueller F, et al. (2019) Evaluating burst buffer placement in HPC systems. In: Proceeding of the 2019 IEEE International Conference on Cluster Computing (CLUSTER), Albuquerque, NM, 23–26 September 2019, pp. 1–11. Washington, DC: IEEE.
- Krishnaswamy S, Loke SW and Zaslavsky A (2004) Estimating computation times of data-intensive applications. *IEEE Distributed Systems Online* 5(4): 1.
- Lockwood GK, Snyder S, Wang T, et al. (2018) A year in the life of a parallel file system. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC), Denver, CO, 12–17 November 2017, pp. 74. Manhattan, NY: IEEE Press.
- Lofstead J, Zheng F, Liu Q, et al. (2010) Managing variability in the IO performance of petascale storage systems. In: Proceedings of the 22nd ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis, SC, 13–19 November 2010. ISBN 978-1-4244-7559-9, pp. 1–12.
- Martinasso M and Mehaut JF (2011) A contention-aware performance model for HPC-based networks: a case study of the InfiniBand network. In: Proceedings of the European Conference on Parallel Processing, Warsaw, Poland, 24–28 August 2011, pp. 91–102. Berlin, Germany: Springer.
- Matsunaga A and Fortes JA (2010) On the use of machine learning to predict the time and resources consumed by applications. In: Proceedings of the 10th IEEE/ACM International Conference on Cluster, Cloud and Grid Computing (CCGrid), Melbourne, VIC, 17–20 May 2010, pp. 495–504.
- McKenna R, Herbein S, Moody A, et al. (2016) Machine learning predictions of runtime and IO traffic on high-end clusters. In: Proceedings of the 2016 IEEE International Conference on

- Cluster Computing (CLUSTER), Taipei, Taiwan, pp. 255–258. Washington, DC: IEEE Computer Society. DOI: [10.1109/CLUSTER.2016.58](https://doi.org/10.1109/CLUSTER.2016.58).
- Mikolov T, Sutskever I, Chen K, et al. (2013) Distributed representations of words and phrases and their compositionality. In: Proceedings of the 26th Neural Information Processing Systems Conference, Lake Tahoe, Nevada, 5–10 December 2013, pp. 3111–3119.
- Miller M (2015) *Design and Implementation of MACSio*. Technical report. Livermore, CA: Lawrence Livermore National Lab.(LLNL).
- Mohammed A, Eleliemy A, Ciorba FM, et al. (2019) An approach for realistically simulating the performance of scientific applications on high performance computing systems. *Future Generation Computer Systems* 111: 617–633.
- Mubarak M, Carns P, Jenkins J, et al. (2017) Quantifying I/O and communication traffic interference on dragonfly networks equipped with burst buffers. In: Proceedings of the 2017 IEEE International Conference on Cluster Computing (CLUSTER), Honolulu, HI, 5–8 September 2017, pp. 204–215. Washington, DC: IEEE.
- Mubarak M, Carothers CD, Ross RB, et al. (2016) Enabling parallel simulation of large-scale HPC network systems. *IEEE Transactions on Parallel and Distributed Systems* 28(1): 87–100.
- Neuwirth S, Wang F, Oral S, et al (2017) Automatic and transparent resource contention mitigation for improving large-scale parallel file system performance. In: Proceedings of the 2017 IEEE 23rd International Conference on Parallel and Distributed Systems (ICPADS), Shenzhen, China, 15–17 December 2017, pp. 604–613. Washington, DC: IEEE.
- Neuwirth S, Wang F, Oral S, et al. (2016) Using balanced data placement to address I/O contention in production environments. In: Proceedings of the 2016 28th International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD), Los Angeles, CA, 26–28 October 2016, pp. 9–17. Washington, DC: IEEE.
- Nurmi D, Brevik J and Wolski R (2007) QBETS: queue bounds estimation from time series. In: Proceedings of the the Workshop on Job Scheduling Strategies for Parallel Processing, Saint-Malo, France, 26 June 2006, pp. 76–101.
- Rodrigues ER, Cunha RL, Netto MA, et al. (2016) Helping HPC users specify job memory requirements via machine learning. In: Proceedings of the Third International Workshop on HPC User Support Tools, Salt Lake City, UT, 13 November 2016, pp. 6–13.
- Smith W, Foster I and Taylor V (1998) Predicting application runtimes using historical information. In: Proceedings of the Workshop on Job Scheduling Strategies for Parallel Processing, Orlando, FL, 30 March 1998, pp. 122–142.
- Smith W, Taylor V and Foster I (1999) Using run-time predictions to estimate queue wait times and improve scheduler performance. In: Proceedings of the Workshop on Job Scheduling Strategies for Parallel Processing, San Juan, Puerto Rico, 16 April 1999, pp. 202–219.
- Sung H, Bang J, Sim A, et al. (2019) Understanding parallel I/O performance trends under various HPC configurations. In: Proceedings of the ACM Workshop on Systems and Network Telemetry and Analytics, Phoenix, AZ, 25 June 2019, pp. 29–36.
- Tsafrir D, Etsion Y and Feitelson DG (2007) Backfilling using system-generated predictions rather than user runtime estimates. *IEEE Transactions on Parallel and Distributed Systems* 18(6): 789–803.
- Wyatt MR, Herbein S, Ahn D, et al. (2017) *Unstructured Data Analytics for Next-Generation HPC Schedulers: Capturing Jobs' Needs from Unstructured Job Scripts*. Technical report. Livermore, CA: Lawrence Livermore National Lab.(LLNL).
- Wyatt MR, Herbein S, Gamblin T, et al. (2018) PRIONN: predicting runtime and IO using neural networks. In: Proceedings of the 47th International Conference on Parallel Processing, (ICPP), Eugene, OR, p. 46:1–46:12. New York, NY: ACM. DOI: [10.1145/3225058.3225091](https://doi.org/10.1145/3225058.3225091).
- Wyatt MR, Herbein S, Shoga K, et al. (2020) CanarIO: sounding the alarm on IO-related performance degradation. In: Proceedings of the 2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS), New Orleans, LA, 13–16 August 2018. IEEE Computer Society, pp. 73–83. DOI: [10.1109/IPDPS47924.2020.00018](https://doi.org/10.1109/IPDPS47924.2020.00018).
- Xu Y, Arteaga D, Zhao M, et al (2012) vPFS: bandwidth virtualization of parallel storage systems. In: Proceedings of the IEEE 28th Symposium on Mass Storage Systems and Technologies (MSST), Pacific Grove, CA, 16–20 April 2012, pp. 1–12. Washington, DC: IEEE.
- Zhao M and Xu Y (2019) vPFS+: managing I/O performance for diverse HPC applications. In: Proceedings of the 2019 35th Symposium on Mass Storage Systems and Technologies (MSST), Santa Clara, CA, 24 May 2019, pp. 51–64. Washington, DC: IEEE.

## Author biographies

*Michael R Wyatt II* is a Scientific Researcher in the Center for Applied Scientific Computing at Lawrence Livermore National Laboratory. His research focuses on high performance computing, machine learning, and resource-aware scheduling. He earned his PhD degrees in Computer Science from the University of Delaware in 2020.

*Stephen Herbein* is a Senior Systems Software Engineer at NVIDIA. His research interests include batch job scheduling, HPC+Cloud convergence, and parallel IO. He earned his PhD, MS, and BS degrees in Computer Science from the University of Delaware in 2018, 2016, and 2014. He is a member of both the IEEE and the ACM.

*Todd Gamblin* is a Computer Scientist in the Center for Applied Scientific Computing at Lawrence Livermore National Laboratory. His research focuses on scalable tools for measuring, analyzing, and visualizing parallel performance data. He is the creator of Spack, a popular HPC

package management tool, and he leads the Software Packaging Technologies area in the U.S. Exascale Computing Project. He received PhD and MS degrees in Computer Science from the University of North Carolina at Chapel Hill in 2009 and 2005.

*Michela Taufer* is an ACM Distinguished Scientist and holds the Jack Dongarra professorship in high performance

computing with the Department of Electrical Engineering and Computer Science, University of Tennessee Knoxville. Her research interests include high performance computing, volunteer computing, scientific applications, scheduling and reproducibility challenges, and in situ data analytics. Dr Taufer received her PhD in Computer Science from the Swiss Federal Institute of Technology (ETH) in 2002.